

Visuelle Programmiersprachen: Low- Code (4)

Franz Höpfinger

2022-2025, Meisterschulen am Ostbahnhof - München

Inhaltsverzeichnis

1. Wiki 4: Visuelle Programmiersprachen	8
1.1 Meisterschulen am Ostbahnhof, München	8
1.2  Einstieg	8
1.3  Kernkonzepte	8
1.3.1 IEC 61499 & 4diac	8
1.4  Ein wenig Inspiration	9
1.5  Navigation	9
2. FunctionBlock	11
3. Literatur zu IEC 61499	0
3.1 Deutsch	0
3.2 Englisch	0
3.3  Podcast	0
4. Übersicht Visuelle Programmiersprachen	0
4.1 1. Erziehungsorientierte und allgemeine Sprachen	0
4.2 2. Visuelle Modellierung in der Informatik	0
4.3 3. Ingenieurswissenschaftliche Sprachen (Datenfluss)	0
4.4 4. Automatisierungstechnik (Industrieller Standard)	0
4.4.1 Die Sprachen der IEC 61131-3	0
4.4.2 Die Evolution: IEC 61499	0
4.5 Warum visuelle Programmierung in der Industrie?	0
4.5.1 Weiterführende Ressourcen	0
5. Welcome	0
6. Übungsaufgaben	0
6.1 Die Aufgaben bauen aufeinander auf, so dass es sich empfiehlt diese in der angegebenen Reihenfolge zu bearbeiten.	0
6.1.1 Eingänge IX / QX (alles tastend)	0
6.1.2 Logik	0
6.1.3 Flip Flops I	0
6.1.4 Rendezvous und Flip-Flop	0
6.1.5 Flip Flops II	0
6.1.6 Flip Flops und Equals	0
6.1.7 Flip Flops und Logikfunktionen	0
6.1.8 Flip Flops und E Funktionen	0
6.1.9 Flip Flops und led_strip	0

7. 4diac	0
7.1 Eclipse 4diac™	0
7.1.1  Video	0
7.1.2  Podcast	0
7.1.3 Kapitelinhalt	0
7.2 Eclipse 4diac™ IDE Icon	0
7.2.1 Function Blocks	0
7.2.2 System	0
7.2.3 Device	0
7.2.4 Event/Data/Adapters	0
7.2.5 ECC	0
7.3 Eclipse 4diac™ Update	0
7.3.1 Update	0
7.3.2 Update URLs for 4Diac	0
7.4  Installation Eclipse 4diac™	0
7.5 Über Eclipse 4diac™	0
7.5.1 Gedicht über 4diac im Stile Göthes	0
7.5.2 xx	0
7.5.3 Geschichte:	0
7.5.4 Open Source	0
7.6 Farben	0
7.6.1 Eclipse 4diac Farbreferenz	0
8. Abgrenzung	0
8.1 Abgrenzung und Einordnung	0
8.1.1 Abgrenzung visuelle/textuelle Programmiersprachen	0
8.1.2 Fokus: Industrielle visuelle Programmiersprachen	0
8.1.3 Zusammenfassung: Warum visuell programmieren?	0
9. Allgemeines	0
9.1  Allgemeines	0
9.1.1 Identifikatoren (Namenskonventionen)	0
9.1.2 Zahlen und Literale	0
9.1.3  Datentypen	0
9.2  Funktionsbausteine in IEC 61499	0
9.2.1 Aufbau eines Funktionsbausteins	0
9.2.2 Typen von Funktionsbausteinen	0
9.2.3  ECC (Execution Control Chart)	0
9.3  Unterschied zwischen IEC 61131-3 und IEC 61499	0
9.3.1 1. Architekturmodell	0

9.3.2	2. Ausführungsmodell	0
9.3.3	3. Funktionsbausteine im Vergleich	0
9.3.4	Zusammenfassung	0
9.4	Zahlen und Zahlensysteme	0
9.4.1	 Podcast	0
9.4.2	Zahlensysteme	0
9.4.3	Literale in der IEC 61131-3 / IEC 61499	0
9.4.4	Mathematische Hintergründe	0
9.5	Datentypen	0
9.5.1	 Datentypen (Übersicht)	0
9.5.2	Typ-Kompatibilität in 4diac-IDE (Datenverbindungen)	0
9.5.3	BOOL	0
9.5.4	BYTE	0
9.5.5	CHAR	0
9.5.6	DATE	0
9.5.7	DATE AND TIME	0
9.5.8	DINT	0
9.5.9	DWORD	0
9.5.10	GenerischeDatentypen	0
9.5.11	INT	0
9.5.12	LDATE	0
9.5.13	LDT	0
9.5.14	LINT	0
9.5.15	LREAL	0
9.5.16	LTIME	0
9.5.17	LTOD	0
9.5.18	LWORD	0
9.5.19	QUARTER	0
9.5.20	REAL	0
9.5.21	SINT	0
9.5.22	STRING	0
9.5.23	TIME	0
9.5.24	TIME OF DAY	0
9.5.25	UDINT	0
9.5.26	UINT	0
9.5.27	ULINT	0
9.5.28	USINT	0
9.5.29	WCHAR	0

9.5.30	WORD	0
9.5.31	WSTRING	0
10.	Bibliotheken	0
10.1	 Bibliotheken	0
10.2	ExternalLibraries	0
10.2.1	External Libraries	0
10.2.2	BlinkMarine	0
10.2.3	DataPanel	0
10.2.4	Funk	0
10.2.5	OSCAT	0
10.2.6	Adapter	0
10.2.7	Iec61131 3 bool	0
10.2.8	Isobus	0
10.2.9	logiBUS	0
10.3	StandardLibraries	0
10.3.1	Standard Libraries	0
10.3.2	Convert	0
10.3.3	Core	0
10.3.4	Events	0
10.3.5	Iec61131 3	0
10.3.6	Io	0
10.3.7	Net	0
10.3.8	Powerlink	0
10.3.9	Reconfiguration	0
10.3.10	Rtevents	0
10.3.11	Signalprocessing	0
10.3.12	System	0
10.3.13	Utils	0
10.4	Vergleich	0
10.4.1	 Vergleich mit anderen Normen	0
10.4.2	IEC61131 3	0
10.5	Typelibrary	0
10.5.1	TypeLibrary	0
10.5.2	MyLib	0
11.	DasErsteProjekt	0
11.1	 Das erste Projekt	0
11.2	4Diac neues Projekt anlegen	0
11.3	Steuerung Bohrmaschinenschalter erstellen	0

11.4	 Video	0
12.	Literatur	0
12.1	Bücher Deutsch	0
12.1.1	IEC 61499 Function Blocks für den Entwurf von Eingebetteten und Verteilten Systemen	0
12.1.2	Prozess-Steuerungen: Projektierung und Inbetriebnahme mit dem Softwaretool SPaS	0
12.2	Bücher Englisch	0
12.2.1	IEC 61499 Function Blocks for Embedded and Distributed Control Systems Design	0
12.2.2	Distributed Control Applications: Guidelines, Design Patterns, and Application Examples with the IEC 61499	0
12.2.3	Modelling Control Systems Using IEC 61499	0
12.3	Sonstiges Deutsch	0
12.3.1	Artikel und Portale	0
12.3.2	UniversalAutomation.org	0
12.4	Sonstiges Englisch	0
12.4.1	Communities und Portale	0
12.4.2	Wissenschaftliche Artikel und Präsentationen	0
12.4.3	Wikipedia und Dokumentationen	0
12.4.4	Buchempfehlungen (Referenzen)	0
12.5	 Videos Deutsch	0
12.5.1	Trend im Engineering - IEC 61131 versus IEC 61499	0
12.5.2	YouTube Kanäle und Playlists	0
12.6	 Videos Englisch	0
12.6.1	Ausgewählte Videos und Playlists	0
13.	Uebungen	0
13.1	test AX	0
13.1.1	Uebungen doc	0
13.2	test B	0
13.2.1	Uebungen doc	0
13.3	test VV	0
13.3.1	Uebungen doc	0
14.	Hw	0
14.1	 Hardware	0
14.1.1	 Podcast	0
14.2	PC	0
14.2.1	PC	0
14.2.2	dual WLAN	0
14.3	SPS	0
14.3.1	SPS	0

14.4	logiBUS	0
14.4.1	logiBUS®	0
14.4.2	DualCAN	0
14.4.3	DualOut	0
14.4.4	Hutschienenmoped	0
15.	Norms	0
15.1	 Normen und Standards	0
15.1.1	IEC 61499: Der Standard für verteilte Systeme	0
15.1.2	IEC 61131-3: Die klassische SPS-Programmierung	0
15.1.3	ISO 11783 (ISOBUS): Kommunikation in der Landtechnik	0
15.1.4	Grafische Symbole und Sicherheit	0
16.	Runtime	0
16.1	 Runtime	0
16.2	Isobus	0
16.2.1	ISOBUS	0
16.2.2	Scaling	0
16.2.3	SoftKey Reduction	0
16.2.4	SoftKeyScaling	0



Meisterschulen am Ostbahnhof

1. Wiki 4: Visuelle Programmiersprachen

1.1 Meisterschulen am Ostbahnhof, München

Willkommen bei der Dokumentation zu visuellen Programmiersprachen.

Diese Dokumentation ist Teil der Wissensdatenbank der Meisterschulen am Ostbahnhof München.

Nützliche Links: * [Hauptmenü](#) * [Super-Suche \(alle Wikis\)](#) * [PDF-Handbuch herunterladen](#)

Willkommen in der Welt der grafischen Programmierung! Diese Dokumentation bietet Ihnen einen umfassenden Einstieg in die Konzepte der visuellen Programmierung, mit einem besonderen Fokus auf die industrielle Automatisierung nach **IEC 61499**.

1.2 Einstieg

Haben Sie sich jemals gefragt, wie man Programme visuell erstellt? Hier finden Sie Ressourcen für Anfänger und Fortgeschrittene – von den Grundlagen in **Blockly** oder **Scratch** bis hin zur professionellen Anwendung in der Industrie.

:::{grid} 2 :gutter: 3

Warum visuelle Programmierung? Abstraktion komplexer Logik in intuitive grafische Bausteine.

Fokus IEC 61499 Der Standard für verteilte, ereignisorientierte Steuerungssysteme. :::

1.3 Kernkonzepte

1.3.1 IEC 61499 & 4diac

Die **IEC 61499** ist eine internationale Norm für die Echtzeit-Verarbeitung in verteilten Steuerungssystemen. Sie bietet eine flexible Plattform und eine gemeinsame Sprache für die Kommunikation zwischen Systemen.

Eclipse 4diac ist die führende Open-Source-Entwicklungsumgebung für diesen Standard. Sie ermöglicht es, komplexe industrielle Anwendungen effizient zu modellieren und zu verteilen.



1.4 🧠 Ein wenig Inspiration

⚠️ Die IEC 61499 – Ein Gedicht im Stile Goethes

Die IEC 61499,
ein System moderner Prägung.
Wie ein Uhrwerk, stets in Line,
gestaltet es die Automatisierung.

In Schichten aufgebaut,
wie ein Turm aus Bauklotzsteinen.
Jeder Baustein gut durchdacht,
um Funktionen zu vereinen.

Wie ein Gedanke, der durch den Kopf geht,
so fließt das Programm durch die Schichten.
Dynamisch, flexibel, stets bereit,
die Systeme zu steuern und zu richten.

1.5 🛠️ Navigation

!!! Dieses Projekt befindet sich in aktiver Entwicklung. Fragen oder Anregungen? Besuchen Sie unser [Discussion Forum](#).

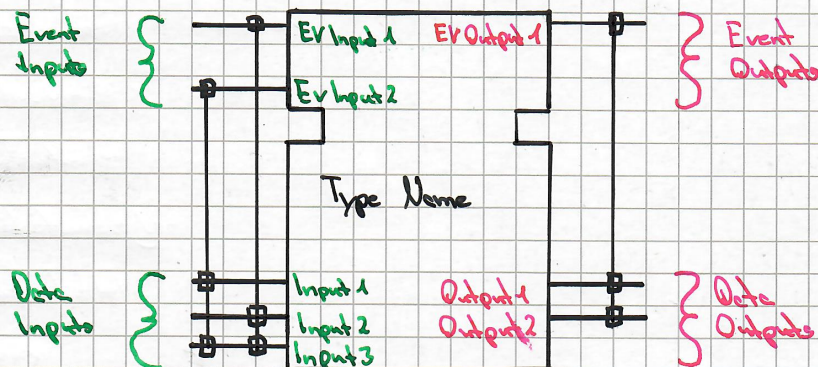
[Hauptmenü](#) | [Super-Suche \(alle Wikis\)](#) | [Schnelle Suche \(IEC 61499\)](#)

2. FunctionBlock

2.0.1 FunctionBlock

Beschreibung Function Block

Der obere Teil wird als Kopf bezeichnet.



Der untere Teil wird als Körper bezeichnet.

- Event Eingänge und Ausgänge sind im Kopf.
- Daten Eingänge und Ausgänge sind im Körper.

Events: Events stehen für die Synchronisation und die Interaktion zwischen den Mechanismen der Ausführungskontrolle der Function Blocks, die im Netzwerk verbunden sind.

Ein Block der Daten sendet, muss ein Event an den Empfängerblock übertragen, um sicherzustellen, dass dieser die Daten liest.

Daten sind typisiert in der Programmiersprache.

Bei den Function Blocks werden Zuordnungen zwischen Events und ^{den Daten} Daten. (vertikale Linien) Die Linien verbinden das Event und seine zugeordneten Daten-Ein- und Ausgänge.

Heißt die Werte der Variablen die mit einem bestimmten Event assoziiert sind, werden beim Eintreten des Events aktualisiert.